Week 5 Part 3 - Tutorial

Finding help and fixing errors

Help files

Every function that we have encountered so far comes with a help file which can be accessed by simply typing `?` followed by the name of the function. For instance typing `?print` in the console will open the help file for the print function. The help files provide details on the inputs, arguments, and outputs of the function, specific details about what is happening under the hood, and examples of the function being used.

Comments

Using comments in your code is an excellent way to remind yourself of what a particular block does. This is helpful when returning to some code after a span of time, or when sharing your code with others. Any text that follows # is treated as a comment and is not run.

```
# This line is not evaluated
print('but this line is')

## [1] "but this line is"
```

Errors

You *will* encounter errors when coding - it is unavoidable. There are myriad reasons for errors to occur - here are some common ones:

1. Typos
2. Missing brackets
3. Using the wrong brackets
4. Not using speech marks for text
5. Variables having not been declared
6. Missing an argument from a function
7. Stray commas
8. Misusing special characters

Let’s see what happens in each of these cases - run each through the console:

```
## 1
primt(hello)
## 2
print "hello"
## 3
print{hello}
## 4 and 5
print(hello)
## 6
print()
# 7
print(c('hello','world',))
# 8
hello% <- 'world'
```

Now try and fix the above lines.

The following questions show a faulty line of code - can you identify the fault?

Quiz

Summary

Here's a summary of the last two worksheets. We have:

1. looked at variables and data types
2. seen how to use and write functions
3. learned how to run code conditionally
4. learned how to loop through a set of instructions several times
5. loaded and saved data to disk

Now we're going to put all of these skills together in order to solve an otherwise tedious task.

Putting it All Together

Seven students submitted data-sets from the same attitudes scale to an open repository.

We would like to take a look at the data, but don't want to have to open each individual file. It is therefore up to you to combine these data-sets into a single file.

Note that some of the data is corrupt, and if it is retained, might affect the results of subsequent analyses.

Starting points

You have been given a base-r cheat-sheet which summarises most of the commands needed for this task.

There is a particularly useful function for this sort of task called `list.files()` which returns the paths to all files in a directory.

Take a look at the help file to see how to use it.

Try opening one data-set to see how it is structured and then think about how you could automate the process.

Questions

1. How many items are there in the attitudes scale?
2. How many data-sets are corrupt?
3. How many observations are there in total?
4. Who collected the most data?

Extra Tasks

1. These students were the first to finish data collection. There are another 40 or so who are still collecting data. How might you adapt your code so that you could handle a data-set of any size?
2. Can you calculate total score for each participant on the attitude scale?
3. If you can do the above, what is the average attitude score across all data-sets?
4. What about the standard deviation?